“Operating System Design and Implementation”

(Chapter 6)
Real Time System and Embedded System

Contents

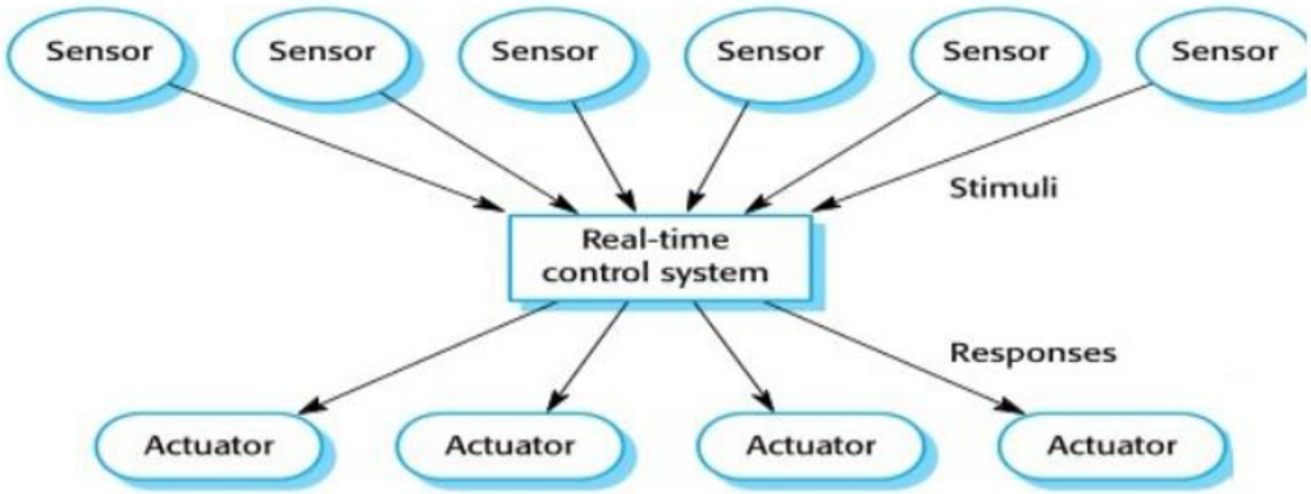
This Chapter will cover:

- Real-Time Scheduling Algorithms (EDF, RMS, RR,LLF)
- Embedded OS Features: TinyOS, FreeRTOS
- Energy-efficient OS Design
- Case Study: Android and Embedded Linux

Real Time System

Introduction

- A real-time system is a computing system that must process and respond to inputs within a specified time constraint, often in milliseconds or microseconds.
- Real time operating system (RTOS) has well defined fixed time limit i.e. processing must be done within the defined time limit, otherwise the system will fail.
- These systems are used in applications where timing is crucial, such as embedded systems, industrial automation, medical devices, and telecommunications.



Real Time System

Types

1. Hard Real-Time Systems
    - Must meet strict deadlines; missing a deadline can lead to failure.
    - Example: Airbag deployment in cars, pacemakers.
2. Soft Real-Time Systems
    - Can tolerate occasional deadline misses, but performance degrades if delays occur frequently.
    - Example: Video streaming, online gaming.

Real Time System

Characteristics of Real-Time Systems

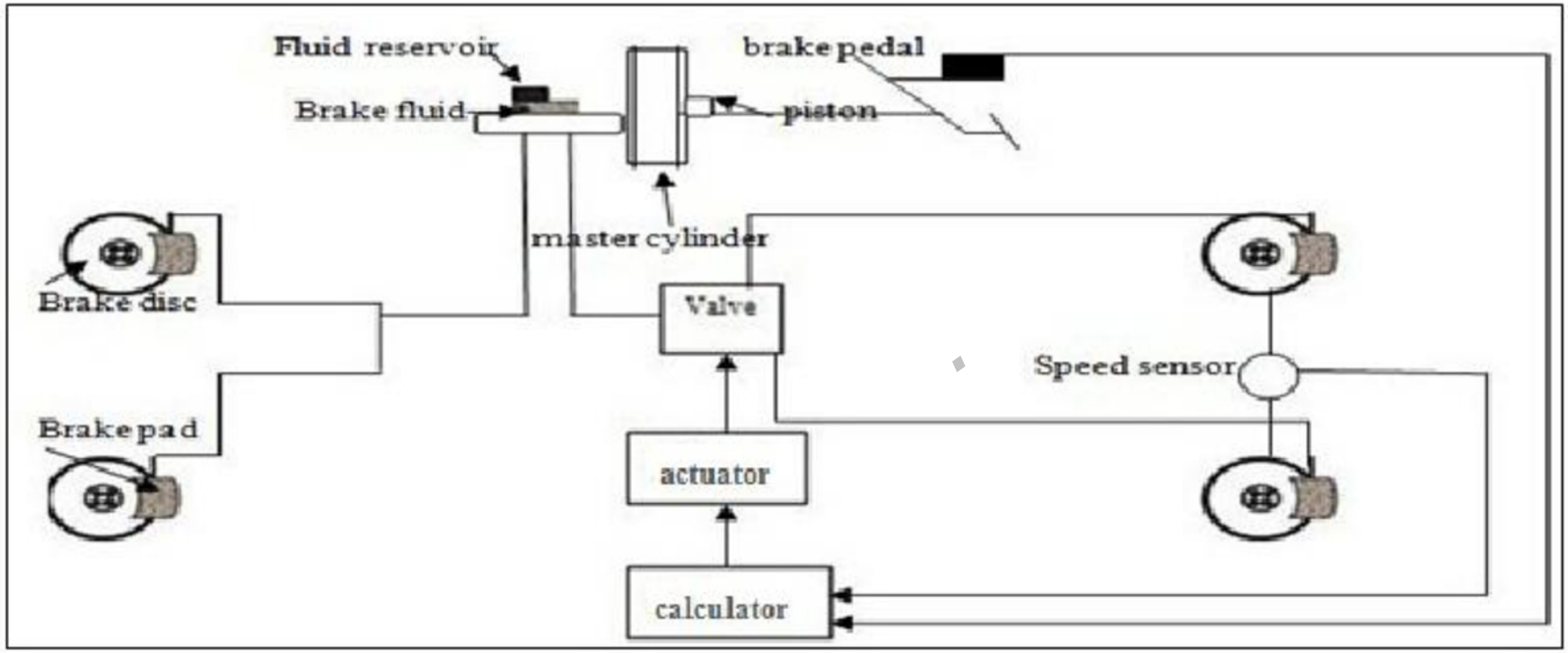
- Timeliness: Responses must occur within the defined time constraints.
- Determinism: System behavior should be predictable.
- Reliability: Must function correctly under all conditions.
- Concurrency: Often handles multiple tasks simultaneously.
- Embedded Nature: Often integrated into larger hardware systems.

Examples of Real-Time Systems

- Flight control systems
- Robotics and automation
- Medical monitoring devices
- Traffic signal control systems
- Fuel Injection System
- Antilock Braking System

Real Time System

Example: Antilock Braking System



Real Time System

Example: Anti-Lock Braking System

- ABS is a hard real-time system that ensures vehicle safety by continuously monitoring and adjusting braking force within strict time limits. It is used in vehicles to prevent wheels from locking up during sudden braking. It ensures vehicle stability and control by rapidly adjusting brake pressure. The system operates under strict time constraints—if it fails to respond within milliseconds, the vehicle may skid, leading to accidents.

Main Components

1. **Brake Pedal & Piston**
   - The driver presses the brake pedal, applying force on the piston to activate the braking system.
2. **Fluid Reservoir & Brake Fluid**
   - Stores and supplies hydraulic brake fluid to transfer braking force from the master cylinder to the brake pads.

Real Time System

Example: Antilock Braking System

3. **Master Cylinder**
   - Converts the mechanical force from the brake pedal into hydraulic pressure, sending brake fluid to the braking components.

4. **Brake Disc & Brake Pads**
   - These components physically create friction to slow down or stop the wheels. The brake pads press against the rotating brake disc to generate stopping force.

5. **Speed Sensor**
   - Continuously monitors wheel rotation speed. Detects if any wheel is about to lock up (i.e., stops rotating while the car is still moving).

6. **Calculator (Electronic Control Unit - ECU)**
   - The brain of the ABS system. Processes real-time data from the speed sensors and calculates whether a wheel is locking up. Sends control signals to adjust brake pressure accordingly.

Real Time System

Example: Antilock Braking System

7. Actuator

- Receives control signals from the ECU and modulates brake force by controlling hydraulic pressure. Ensures that braking is applied efficiently without causing wheel lock-up.

8. Valve

- Regulates the brake fluid pressure to increase or decrease braking force. Works under ECU control to adjust braking pressure dynamically.

- So in this ABS system, speed sensors continuously send real-time wheel speed data to the ECU. The ECU detects if a wheel is decelerating too quickly (risk of locking).If a lock-up is imminent, it decides how much brake pressure to reduce. The actuator and valve adjust brake pressure dynamically to prevent lock-up. This process happens within milliseconds to ensure stability and control. The system continuously monitors wheel speeds and re-adjusts braking force in real-time, repeating the cycle several times per second.

Real Time System

Real Time Scheduling

- Real-time scheduling is the process of managing task execution in real-time systems to ensure that all critical tasks meet their deadline constraints.
- Since real-time systems often operate under strict timing requirements, the scheduler plays a crucial role in determining which task gets CPU time and when.
- Example: In an ABS multiple real-time tasks need to be scheduled to ensure safe and efficient braking. These tasks must be executed concurrently and within strict deadlines to prevent wheel lock-up and maintain vehicle stability.

Key Tasks That Require Scheduling in ABS:

- Wheel Speed Monitoring Task (Sensor Reading) [High Priority]
  - ✓ Reads real-time data from wheel speed sensors.
  - ✓ Must execute every few milliseconds to detect skidding.
- Brake Pressure Calculation Task (Control Algorithm) [High Priority]
  - ✓ The ECU (Electronic Control Unit) processes speed sensor data.
  - ✓ Determines whether a wheel is locking and calculates the required brake force.

Real Time System

Real Time Scheduling

Key Tasks That Require Scheduling in ABS:

- Brake Actuation Task (Valve & Actuator Control) [High Priority]
  - ✓ Sends signals to the hydraulic valves and actuators to adjust braking force.
  - ✓ Must respond instantly to prevent wheel lock-up.

- System Health Monitoring Task [Medium Priority]
  - ✓ Continuously checks ABS components for faults or failures.
  - ✓ Runs periodically but does not require immediate execution.

- Communication Task (Data Logging & Dashboard Alerts) [Low Priority]
  - ✓ Sends ABS status updates to the vehicle dashboard.
  - ✓ Logs braking system performance for diagnostics.

Real Time System

Why Scheduling is Necessary?

- Real-time scheduling ensures that high-priority tasks (like sensor reading and brake actuation) are executed first within strict time constraints.
- Lower-priority tasks (like logging and health monitoring) are scheduled efficiently without delaying critical braking functions.
- A preemptive real-time scheduling approach (e.g., Rate Monotonic Scheduling or Earliest Deadline First) ensures that braking control always gets CPU priority over non-essential tasks.

Real Time System

Real-time scheduling Algorithm: Static Priority Scheduling Algorithm

- Static priority scheduling is a scheduling method where task priorities are assigned before execution begins and remain unchanged throughout the system's operation.
- These priorities are typically determined based on predefined criteria such as task periods, deadlines, or criticality.
- One of the most widely used static priority scheduling algorithm is **Rate Monotonic Scheduling (RMS).**

Real Time System

Real-time scheduling Algorithm: Rate Monotonic Scheduling (RMS).

- Rate monotonic scheduling is a priority algorithm that belongs to the static priority scheduling category of Real time Operating Systems.
- The priority is decided according to the cycle time or period of the processes that are involved. **If the process has a small period meaning that it need to run more frequent, then it has the highest priority. The task execution time or absolute deadline is NOT considered in priority assignment.**
- It is preemptive in nature. Thus, if a process with highest priority starts execution, it will preempt the other running processes. The priority of a process is inversely proportional to the period it will run for.
- A set of processes can be scheduled only if they satisfy the following equation:

Real Time System

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq n(2^{1/n} - 1)$$

Where n is the number of processes in the process set, Ci is the computation time of the process, Ti is the Time period for the process to run and U is the processor utilization.

- When CPU utilization (U) is 0% is when a real-time system is completely idle, meaning the CPU is not executing any tasks (ci=0). If all available CPU time is spent on executing tasks, it leads to 100% CPU utilization.

Real Time System

CPU Utilization

- CPU utilization in a real-time system refers to the percentage of CPU time spent executing real-time tasks versus remaining idle. It plays a critical role in ensuring that time-sensitive tasks meet their deadlines while optimizing resource usage.
- The idea of real-time scheduling is to confine processor utilization under schedulable utilization of a certain scheduling algorithm, which is scaled from 0 to 1. Higher schedulable utilization means higher utilization of resource and the better the algorithm.
- Formula for CPU Utilization in Real-Time Systems
- For n periodic tasks in a real-time system, CPU utilization is given by:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i}$$

Where:

Ci = Computation (execution) time of task iii

Ti = Period of task I

- CPU utilization in real-time systems ensures timely task execution, system stability, schedulability, efficient resource usage, and predictable performance for critical applications.

Real Time System

Consider three periodic tasks:

Task	Execution Time (C_i)	Period (T_i)
T_1	1 ms	4 ms
T_2	2 ms	5 ms
T_3	1 ms	10 ms

Now, calculate CPU utilization:

$$U = \frac{1}{4} + \frac{2}{5} + \frac{1}{10}$$

$$U = 0.25 + 0.4 + 0.1 = 0.75$$

Real Time System

- **Schedule the below task using RMS. Also Calculate Average Waiting Time, Average Turnaround Time and Average Response Time.**

Processes	Execution Time (C)	Time period (T)
P1	3	20
P2	2	5
P3	2	10

Note: if no Deadline means Period(ending time) =Deadline and similarly if no Release Time means Period(starting value)=Release time

Solution:

- These process can be scheduled if they satisfy the following combined utilization equation

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq n(2^{1/n} - 1)$$

Real Time System

- n(2^1/n - 1) = 3 (2^1/3 - 1) = 0.7977
- Combined Utilization (U) = 3/20 + 2/5 + 2/10 = 0.75 (i.e. u<=1)

Since 0.75<=1(Necessary Condition) and 0.75<=0.79(Sufficient Condition), The above set of process/Task can be scheduled under RMS.

Now Scheduling Time/Hyper period=?

Since LCM(20,5,10)=20, All these processes can be scheduled in 20-time units. Further Process should be scheduled as per priority set under the smaller time period. The priority will be the highest for the process which has the least running time period. Thus P2 will have the highest priority, and after that P3 and lastly P1.

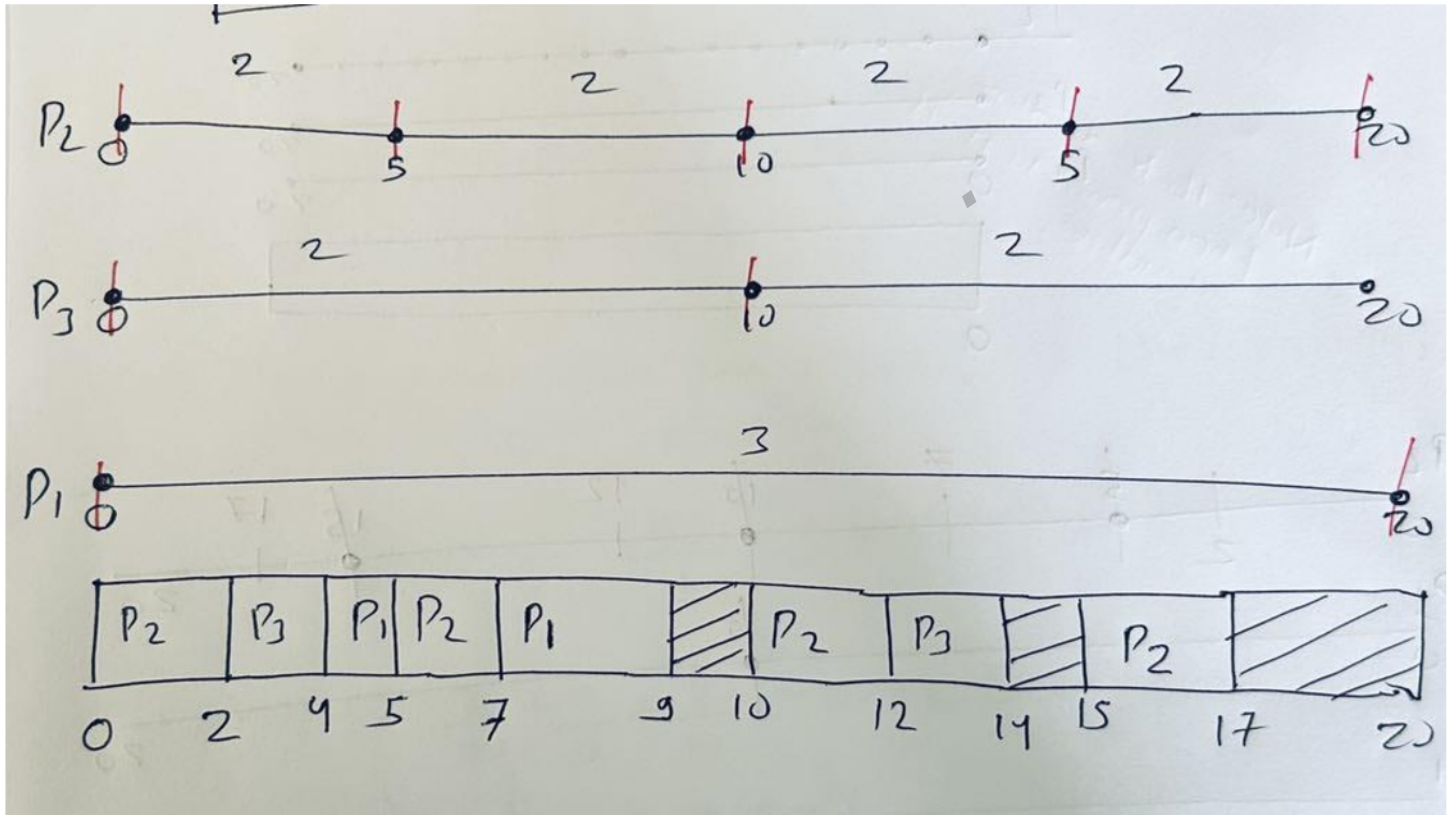
P2> P3>P1

Real Time System

Assumptions:

- ●: Period
- | : Deadline

Processes	Execution Time (C)	Time period (T)
P1	3	20
P2	2	5
P3	2	10



Real Time System

- The above figure says that Process P2 will execute two times for every 5 time units, Process P3 will execute two times for every 10 time units and Process P1 will execute three times in 20 time units.
- Process P2 will run first for 2 time units because it has the highest priority. After completing its two units, P3 will get the chance and thus it will run for 2 time units. As we know that process P2 will run 2 times in the interval of 5 time units and process P3 will run 2 times in the interval of 10 time units, they have fulfilled the criteria and thus now process P1 which has the least priority will get the chance and it will run for 1 time. And here the interval of five time units have completed. Because of its priority P2 will preempt P1 and thus will run 2 times. As P3 have completed its 2 time units for its interval of 10 time units, P1 will get chance and it will run for the remaining 2 times, completing its execution which was thrice in 20 time units. Now 9-10 interval remains idle as no process needs it. At 10 time units, process P2 will run for 2 times completing its criteria for the third interval (10-15). Process P3 will now run for two times completing its execution. Interval 14-15 will again remain idle for the same reason mentioned above. At 15 time unit, process P2 will execute for two times completing its execution. This is how the rate monotonic scheduling works.

Real Time System

Rate monotonic scheduling algorithm

Advantages:

- It is easy to implement.
- It consists of a calculated copy of the time periods, unlike other time-sharing algorithms as Round robin which neglects the scheduling needs of the processes.

Disadvantages:

- It is very difficult to support aperiodic tasks under RMA. RMA assigns priority based on fixed periods (shorter periods → higher priority).
- RMA is not optimal when the task period and deadline differ. Rate Monotonic Algorithm (RMA) assumes that the deadline of a task is equal to its period (Di=Ti). It assigns priorities based solely on task periods, with shorter-period tasks getting higher priority. However, when the deadline Di is different from the period, RMA is no longer optimal because it may fail to schedule all tasks even if a feasible schedule exists.

Real Time System

Rate monotonic scheduling algorithm

Example Where RMA Fails but Another Scheduling Works?

Consider two tasks:

- Task 1: C1=2 , T1=5, D1=4
- Task 2: C2=3, T2=7, D2=7

- RMA gives Task 1 higher priority since T1<T2, However, Task 2 has a more relaxed period but a critical deadline, leading to a potential deadline miss. Here actually Task 2 is more critical because of its strict deadline (D2 = 7 with Period T1= 7), while Task 1's earlier deadline (D1 = 4 with Period T1=5) allows for more room to complete the task within the period without missing the deadline. If Task 2 takes too long to complete, or if it is delayed by higher-priority tasks (like Task 1), there is a higher chance that Task 2 will miss its deadline.
- Earliest Deadline First (EDF) dynamically adjusts priorities based on deadlines would work better in this scenario and is optimal for any set of periodic tasks (as long as CPU utilization U≤100%).

Real Time System

Real-time scheduling Algorithm: Dynamic Priority Scheduling Algorithm

- Dynamic priority scheduling is a type of scheduling algorithm in which the priorities are calculated during the execution of the system.
- The goal of dynamic priority scheduling is to adapt to dynamically changing progress and to form an optimal configuration in a self-sustained manner.
- Unlike fixed priority, dynamic priority scheduling could dynamically prioritize task deadlines achieving optimal schedulable utilization in the preemptible case.

Why Dynamic Priority Scheduling?

- Real-time processing requires prioritization based on deadlines.
- Fairness is needed to prevent starvation of low-priority processes.

Types

- Earliest Deadline First Scheduling
- Least Slack Time Scheduling

Real Time System

Real-time scheduling Algorithm: Earliest Deadline First (EDF)

- Earliest deadline first (EDF) or least time to go is a dynamic priority scheduling algorithm used in real time operating systems to place processes in a priority queue.
- Whenever a scheduling event occurs (task finishes, new task released, etc.) the queue will be searched for the process closest to its deadline. This process is the next to be scheduled for execution.
- EDF uses priorities to the jobs for scheduling. It assigns priorities to the task according to the absolute deadline. The task whose deadline is closest gets the highest priority. The priorities are assigned and changed in a dynamic fashion. EDF is very efficient as compared to other scheduling algorithms in real-time systems. It can make the CPU utilization to about 100% while still guaranteeing the deadlines of all the tasks. In EDF, if the CPU usage is less than 100%, then it means that all the tasks have met the deadline. EDF finds an optimal feasible schedule. The feasible schedule is one in which all the tasks in the system are executed within the deadline.

Real Time System

Real-time scheduling Algorithm: Earliest Deadline First (EDF)

- If EDF is not able to find a feasible schedule for all the tasks in the real-time system, then it means that no other task scheduling algorithms in real-time systems can give a feasible schedule.
- EDF scheduling algorithm does not need the tasks or processes to be periodic and also the tasks or processes require a fixed CPU burst time.
- In EDF, any executing task can be preempted if any other periodic instance with an earlier deadline is ready for execution and becomes active. Preemption is allowed in the Earliest Deadline First scheduling algorithm

Real Time System

Real-time scheduling Algorithm: Earliest Deadline First (EDF)

Condition for Tasks to be scheduled in EDF

- For **preemptive EDF scheduling**, a set of periodic tasks is schedulable if the total CPU utilization **U** satisfies:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \leq 1$$

where:

- Ci is the computation (execution) time of task i,
- Ti is the period of task I

Note that.

- If a feasible schedule exists, EDF can always find it.
- Unlike fixed-priority scheduling (like RMS), EDF can schedule tasks up to full CPU usage.
- If CPU utilization exceeds 100%, some deadlines will miss.

Real Time System

Real-time scheduling Algorithm: Earliest Deadline First (EDF)

Advantage

- Meeting Deadlines: EDF ensures that tasks with the earliest deadlines are executed first. By prioritizing tasks based on their deadlines, EDF minimizes the chances of missing deadlines and helps meet real-time requirements.

- Optimal Utilization: EDF maximizes CPU utilization by allowing tasks to execute as soon as their deadlines arrive, as long as the CPU is available. It optimizes the use of system resources by minimizing idle time.

- Responsiveness: EDF provides a high level of responsiveness for time-critical tasks. It ensures that tasks are scheduled and executed promptly, reducing response times and improving system performance.

- Predictability: EDF provides predictability in terms of task execution times and deadlines. The scheduling decisions are deterministic and can be analyzed and predicted in advance, which is crucial for real-time systems.

- Flexibility: EDF can handle both periodic and aperiodic tasks, making it suitable for a wide range of real-time systems. It allows for dynamic task creation and scheduling without disrupting the execution of existing tasks.

Real Time System

Real-time scheduling Algorithm: Earliest Deadline First (EDF)

Disadvantage

- Transient Overload Problem: EDF works well when CPU utilization is ≤ 100%. However, if tasks temporarily exceed the available CPU capacity (i.e., transient overload), EDF cannot guarantee which tasks will miss their deadlines. Some tasks may miss deadlines unpredictably, causing system failures in hard real-time systems.

- Resource Sharing Problem: EDF assumes tasks are independent (i.e., no resource dependencies). When multiple tasks share a resource (e.g., file access, I/O device, mutex), **priority inversion can occur**. A high-priority task may have to wait for a lower-priority task to release a resource, causing deadline misses.

- Efficient Implementation Problem: EDF dynamically changes task priorities based on deadlines, requiring frequent sorting of the task queue. This increases the scheduling overhead, especially for large task sets.

Real Time System

Along with timeline, show how the following set of tasks be scheduled using EDF. Also Calculate Average Waiting Time, Average Turnaround Time and Average Response Time.

Task	Computation	Deadline	Period
T1	3	7	20
T2	2	4	5
T3	2	8	10

Note: if no Release Time means Period(starting value)=Release time

Solution:

CPU Utilization(U)= 3/20 + 2/5 + 2/10 = 0.75 i.e. 75%

Since u<=1, the system is schedulable under EDF.

Real Time System

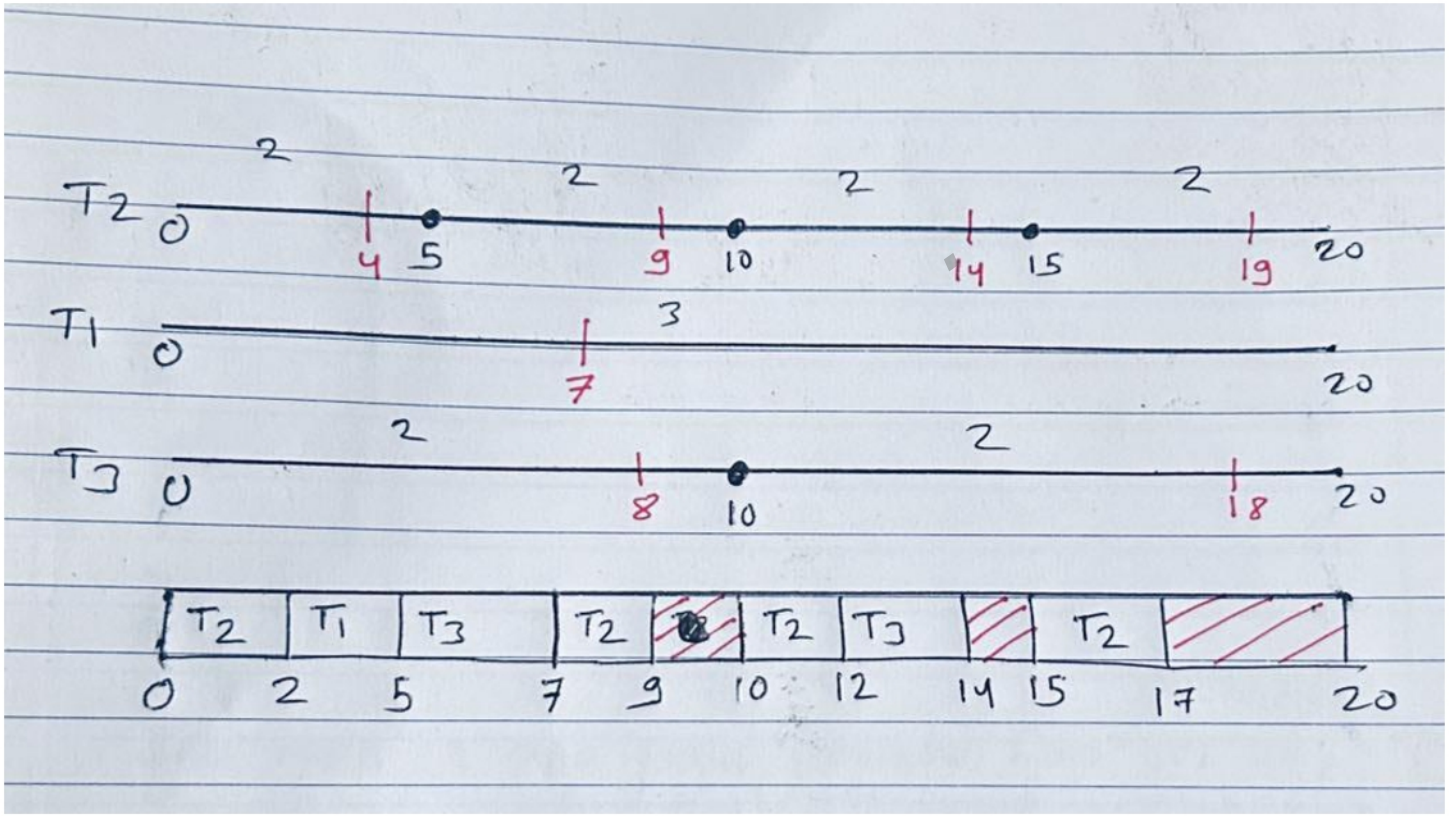
Now Scheduling Time/Hyper period=?

- Since LCM(20, 5, 10)=20, All these processes can be scheduled in 20-time units. Further Process should be scheduled as per priority set under the deadline period. The priority will be the highest for the process which has the least deadline period. Thus initially, T2 will have the highest priority, and after that T1 and lastly T3. However, this priority is dynamically updated throughout the hyper period.

T2>T1>T3

Real Time System

Task	Computation	Deadline	Period
T1	3	7	20
T2	2	4	5
T3	2	8	10



Real Time System

Real-time scheduling Algorithm: Least Laxity First (LLF)

- Least Laxity First (LLF) is a dynamic scheduling algorithm used in real-time systems to schedule tasks based on their laxity (or slack time).
- The task with the smallest laxity is scheduled first.
- **Laxity (Slack Time) Calculation:**

$$\text{Laxity} = (\text{Deadline} - \text{Current Time}) - \text{Remaining Execution Time}$$

This measures how much time is left before a task must complete. A lower laxity means the task is more urgent. A task with zero laxity must execute immediately, or it will miss its deadline.

- LLF is preemptive, meaning a running task can be interrupted if another task's laxity becomes smaller.
- A set of tasks is guaranteed schedulable under LLF only if no task reaches a negative laxity before its deadline.

Real Time System

Real-time scheduling Algorithm: Least Laxity First (LLF)

- In Least Laxity First (LLF) scheduling, laxity (or slack time) is continuously changing as time progresses.
- However, the interval at which laxity should be recalculated depends on the system's scheduling policy and how frequently task priorities need to be updated.
- **Two approaches to calculate laxity**

1. **At Every Scheduling Decision Point**
   - ✓ Laxity is calculated whenever a new task arrives or an executing task completes.
   - ✓ This is useful in event-driven systems.
2. **At Regular Time Intervals (Time-Triggered Systems)**
   - ✓ Laxity can be recalculated at each clock tick (e.g., every 1ms, 10ms, etc.).
   - ✓ Useful for systems with periodic task execution.
   - ✓ This is useful in clock-driven systems or Time Triggered System.

Real Time System

Real-time scheduling Algorithm: Least Laxity First (LLF)

Advantages

- Ensures deadlines are met by always prioritizing the most urgent task.
- Adapts dynamically to changes in execution time.

Disadvantages

- High overhead due to frequent laxity recalculations.
- Jitter issues because tasks get preempted often. Jitter refers to variations in the execution time or response time of tasks due to scheduling interruptions. When tasks frequently get preempted (interrupted to allow another task to run), their execution becomes inconsistent.

Real Time System

Along with timeline, show how the following set of tasks be scheduled using LLF. Also calculate AWT, ATT and ART.

Task	Computation	Deadline	Period
T1	3	7	20
T2	2	4	5
T3	2	8	10

Solution:

CPU Utilization(U)= 3/20 + 2/20 + 2/10 = 0.75 i.e. 75%

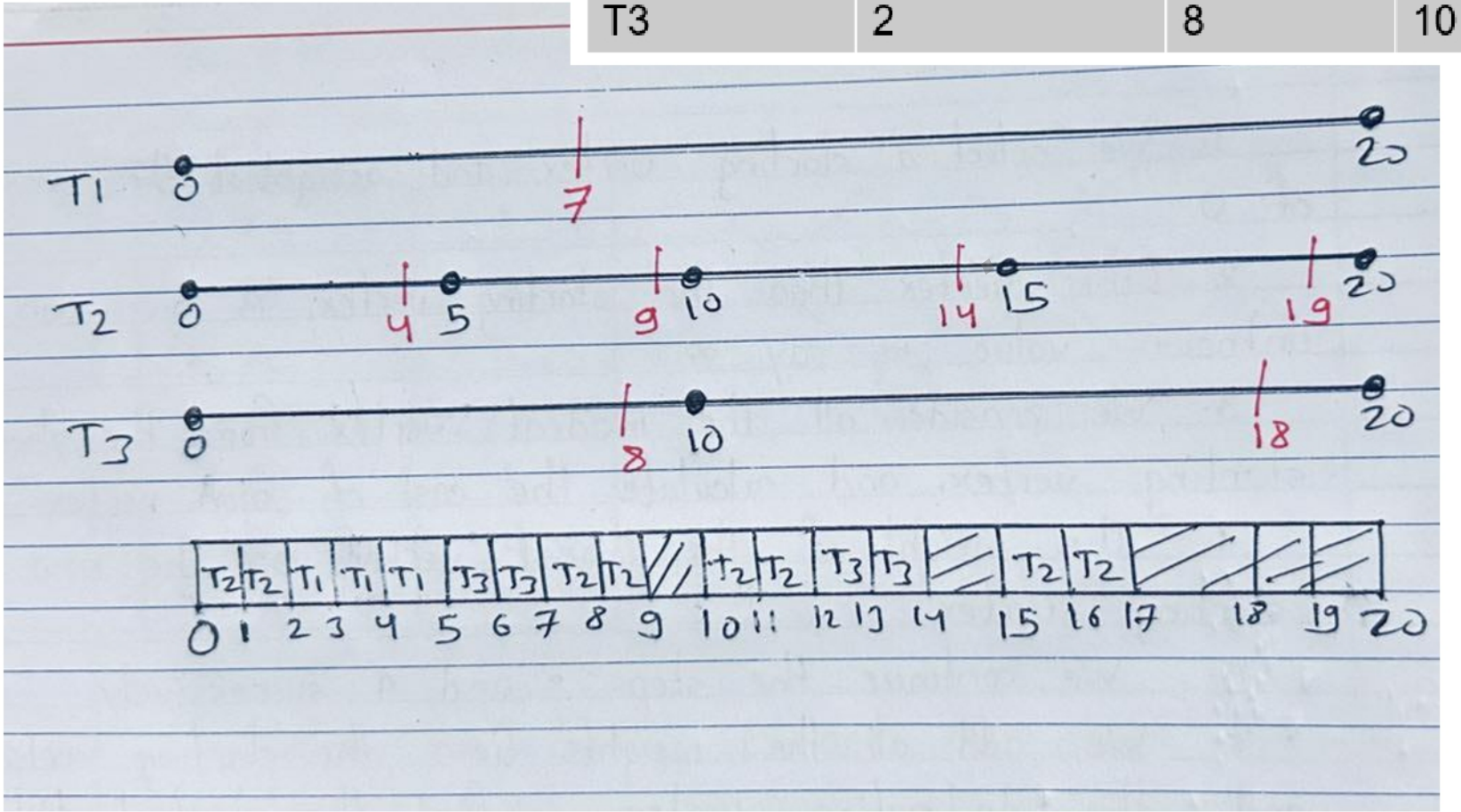
Since u<=1, the system is schedulable under LLF.

Now Scheduling Time/Hyper period=?

- Since LCM(20, 5, 10)=20, All these processes can be scheduled in 20-time units. Further Process should be scheduled as per priority set under the laxity value. The priority will be the highest for the process which has the least laxity.

Real Time System

Task	Computation	Deadline	Period
T1	3	7	20
T2	2	4	5
T3	2	8	10



Real Time System

Calculating Laxity and Updating schedule at an interval of 1 time unit

- Initially, at T=0,

  Laxity of T1=7-0-3=4

  Laxity of T2=4-0-2= 2 (*smallest laxity means highest priority*)

  Laxity of T3= 8-0-2=6

  Thus initially, T2 will have the highest priority

- At T=1,

  Laxity of T1=7-1-3=3

  Laxity of T2=4-1-1=2

  Laxity of T3= 8-1-2=5

  Thus T2 will have the highest priority

Task	Computation	Deadline	Period
T1	3	7	20
T2	2	4	5
T3	2	8	10

Real Time System

Least Laxity First (LLF): Example

- At T=2,

  Laxity of T1=7-2-3=2

  Laxity of T3= 8-2-2=4

  Thus T1 will have the highest priority

- At T=3,

  Laxity of T1=7-3-2=2

  Laxity of T3= 8-3-2=3

  Thus T1 will have the highest priority

Real Time System

Least Laxity First (LLF): Example

- At T=4,

  Laxity of T1=7-4-1=2

  Laxity of T3= 8-4-2=2

  Thus T1 will have the highest priority as T1 and T3 has same laxity and currently T1 is Being Processed

- At T=5,

  Laxity of T2=4-0-2=2 [for T2, next release starts from 5 meaning current time to 0]

  Laxity of T3= 8-5-2=1

  Thus T3 will have the highest priority.

Real Time System

Least Laxity First (LLF): Example

- At T=6,

  Laxity of T2=4-1-2=1 [for T2, next release starts from 5 meaning current time to 0]

  Laxity of T3= 8-6-1=1

  Thus T3 will have the highest priority as T2 and T3 has same laxity and currently T3 is Being Processed

- At T=7,

  Candidate is only T2

- At T=8,

  Candidate is only T2

- At T=9,

  No any Task

Real Time System

Least Laxity First (LLF): Example

- At T=10,

  Laxity of T2=4-0-2=2 [for T2, next release starts from 10 meaning current time to 0]

  Laxity of T3= 8-0-2=6 [for T3, next release starts from 10 meaning current time to 0]

  Thus T2 will have the highest priority

- At T=11,

  Laxity of T2=4-1-1=2 [for T2, next release starts from 10 meaning current time to 0]

  Laxity of T3= 8-1-2=5 [for T3, next release starts from 10 meaning current time to 0]

  Thus T2 will have the highest priority

- At T=12,

  Candidate is only T3

- At T=13,

  Candidate is only T3

Real Time System

Least Laxity First (LLF): Example

- At T=14,

  No any Task

- At T=15,

  Candidate is only T2

- At T=16,

  Candidate is only T2

- At T=17 onwards

  No any Task, CPU Idle

Embedded Operating System

Introduction

- An Embedded System is a **specialized computing system** designed to perform a **specific function or set of functions within a larger system**, often operating under real-time constraints.
- It is a combination of hardware (like microcontrollers) and software (like real-time operating systems) that works autonomously or with minimal user intervention.
- An Embedded Operating System (Embedded OS) is a specialized OS designed to manage hardware resources and run applications in an embedded system. Unlike general-purpose OS (Windows, Linux, macOS), an embedded OS is optimized for performance, small footprint, and real-time constraints.

Embedded Operating System

Introduction

- Microcontrollers drive embedded systems. Microcontroller is ~95% of a computer within a single chip
  - ✓ CPU(16 bit or 32 bit) RAM (~256KB) Flash storage (32 to 2048KB)
  - ✓ Hardware controllers for various signals and buses: Digital I/O, Analog I/O: Ethernet
  - ✓ Radio for wireless communication: Bluetooth Low Energy, WiFi

Characteristics of Embedded Systems

- **Dedicated Functionality/Application Specific:** Designed for a specific task, unlike general-purpose computers.
- **Real-time operation:** Must respond within strict timing constraints.
- **Embedded Software (Firmware):** Runs on microcontrollers or microprocessors, often using an RTOS

Embedded Operating System

Characteristics of Embedded System

- **Interfacing Capabilities:** Communicates with sensors, actuators, and other hardware components.
- **Resource constraints:** Limited memory, processing power, and energy consumption.
- **Reliability:** High dependability, often operating in critical environments.
- **Minimal user interface:** Many embedded systems work autonomously with little to no user interaction.

Embedded Operating System

Application Areas of Embedded System

- Consumer Electronics:
  - ✓ Smartphones & Tablets – Touchscreen controllers, power management, sensor
  - ✓ Smart TVs & Set-Top Boxes – Digital signal processing, video decoding
  - ✓ Digital Cameras – Image processing, autofocus control
- Automobile
  - ✓ Engine Control Unit (ECU) – Monitors and optimizes engine performance
  - ✓ Anti-lock Braking System (ABS) – Prevents wheel lockup during braking
  - ✓ Airbag Control System – Deploys airbags in case of a crash
  - ✓ Advanced Driver Assistance Systems (ADAS) – Lane detection, collision warning
  - ✓ Infotainment Systems – Touchscreen controls, GPS navigation
- Industrial Automation
  - ✓ Robotics – Autonomous machines for manufacturing and logistics
  - ✓ CNC Machines – Computerized numerical control for precision machining
- Medical Devices
  - ✓ Pacemakers – Regulates heartbeats in cardiac patients
  - ✓ Wearable Health Monitors – Smartwatches

Embedded Operating System

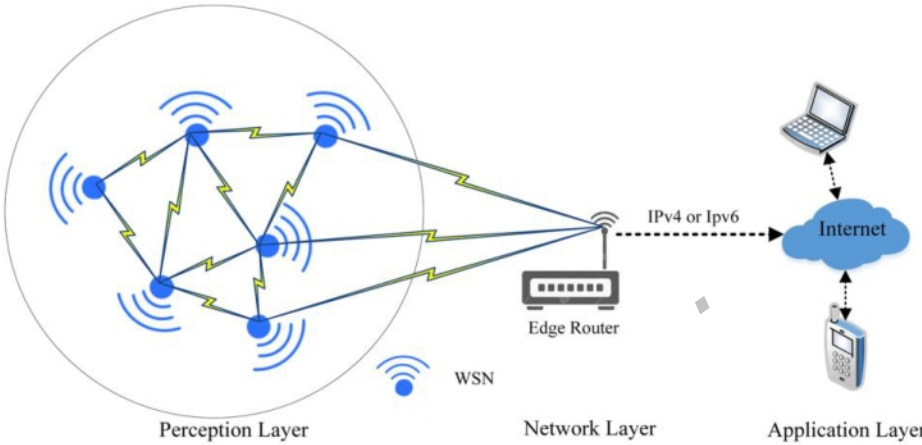
Application Areas of Embedded System

- **Internet of Things (IoT) & Smart Devices**
  - ✓ Smart Homes – Smart lighting, security cameras, voice assistants
  - ✓ Smart Agriculture – Automated irrigation, soil monitoring
- **Telecom:**
  - ✓ Cellular Telephones
  - ✓ Telephone switches,
  - ✓ Handset Multimedia Applications etc.
- **Banking & Retail**
  - ✓ Automatic Teller Machines (ATM) and Currency counters, Point of Sales (POS)
- **Card Readers**
  - ✓ Barcode, Smart Card Readers, Hand held Devices etc.

Real Time System

RTOS: TinyOS, FreeRTOS

- TinyOS is an open-source, event-driven operating system designed for wireless sensor networks (WSNs) and low-power embedded devices.



- TinyOS is optimized for single task, ultra-low-power operation and event-driven processing.
- FreeRTOS is an open-source, real-time operating system designed for embedded devices with preemptive multi-tasking support.
- It is widely used in industrial, automotive, and IoT applications. Best for general purpose RTOS as FreeRTOS supports multi-tasking, preemptive scheduling, and real-time processing.

Real Time System

Energy-efficient OS Design

- Energy efficiency is a critical aspect of modern operating system (OS) design, especially in embedded systems, mobile devices, and data centers.
- The OS plays a significant role in power management, resource allocation, and process scheduling to optimize energy consumption while maintaining system performance.
- An Energy-Efficient OS (EEOS) is designed to minimize power consumption while ensuring optimal performance. This is achieved through:

Real Time System

Energy-efficient OS Design: Key Techniques

- **Dynamic Voltage and Frequency Scaling (DVFS)**
  - ✓ DVFS is a power-saving technique where the processor adjusts its voltage and clock frequency dynamically based on the workload.
  - ✓ Reduces power consumption in idle or low-processing states.
  - ✓ Used in mobile devices, IoT systems, and data centers.

- **Power-Aware Process Scheduling i.e. Efficient Task Scheduling:**
  - ✓ Reducing CPU workload and idle time. Schedulers must balance power efficiency and real-time performance.
  - ✓ Traditional round-robin and FIFO scheduling may not be optimal for energy efficiency.

Real Time System

Energy-efficient OS Design: Key Techniques

- ✓ Advanced techniques:
  - Energy-Aware EDF (Earliest Deadline First) for real-time systems.
  - Power-aware Multi-Core Scheduling to distribute tasks efficiently.

- **Memory and Storage Power Management**
  - ✓ Dynamic RAM power saving by using low-power states (e.g., DDR Self-Refresh).
  - ✓ Flash storage optimization by reducing excessive writes in SSDs to extend lifespan and save power.
  - ✓ Data Compression & Caching: Reducing disk I/O to lower power usage.

- **Device and Peripheral Power Management**
  - ✓ OS must dynamically enable/disable hardware components (Wi-Fi, Bluetooth, GPS, sensors, displays).
  - ✓ Wake-up policies minimize unnecessary activations of devices.

Real Time System

Energy-efficient OS Design: Key Techniques

- **Energy-efficient Networking**
  - ✓ Adaptive Wi-Fi & LTE Power Modes: Reduce energy in idle mode.
  - ✓ Energy-aware Routing Protocols: Used in IoT and sensor networks. Example, IEEE 802.11 Power Save Mode (PSM) extends battery life in Wi-Fi devices.

- Some Energy Efficient OS and their features
  - ✓ Android and IOS: Adaptive Battery, Adaptive Brightness etc.
  - ✓ Windows: Batter Saver Mode

Real Time System

Challenges in Energy-Efficient OS Design

- **Balancing Power & Performance:** Too much power saving can affect system responsiveness.
- **Hardware Dependency:** OS needs hardware support (DVFS, low-power peripherals, sensors).
- **Real-time Constraints:** Embedded and real-time systems must meet deadlines while saving power.
- **Security Concerns:** Aggressive power-saving can lead to side-channel attacks or security loopholes. For example, Side Channel attack (Timing attacks) exploit variations in how long a system takes to perform operations, such as encryption or password checks. In power-saving states, these delays can be used by attackers to deduce sensitive data like passwords or encryption keys.

Real Time System

Further Study

- Microprocessor Vs Microcontroller
- General OS vs RTOS
- Schedule Following Task under RMS T1(C1=1, P1=4), T2(C2=2, P2=5) and T3 (C3=5, P3=20). Also calculate AWT, ART and ATT.
- Schedule Following Task under EDF T1(C1=0.9, P1=2, Deadline=2), T2(C2=2.3, P2=5, Deadline=5) Also calculate AWT, ART and ATT.
- Schedule Following Task under EDF Also calculate AWT, ART and ATT.

Task	Rising Time (Arrival Time)	Execution Time	Deadline	Period
T1	2	2	7	20
T2	1	1	4	5
T3	0	2	8	10

Real Time System

Further Study

- Briefly explain different approaches to Real time scheduling

  (Weighted Round Robin Approach and Priority Driven Approach)

- Differences between TinyRTOS vs FreeRTOS. Also explain the key features of TinyRTOS and FreeRTOS.
- Case Study: Android and Embedded Linux